

AI Enhanced Community Event Platform

Project Details

Role: Fullstack Developer

Project Type: fullstack

Difficulty: junior

Duration: 365 days

Focus: api_development, authentication, database, fullstack, problem_solving, testing, security, microservices, real_time, cloud, performance, system_design, ai_integration, frontend

Student Name: Rahul Sharma

Description: A web platform where organizers create events, attendees book tickets, and an AI service suggests events based on user interests.

Technologies

- JavaScript
- TypeScript
- React
- Next.js
- Node.js
- Express.js
- Laravel
- Tailwind CSS
- MongoDB
- MySQL
- JWT
- AWS
- Python
- FastAPI
- NumPy

Requirements

- Provide role based authentication (admin, organizer, attendee).
- Enable event creation, ticket booking, PDF ticket export, and real time chat.
- Integrate a Python AI service that returns top 3 event recommendations per user.

Objectives

- Implement a secure REST API for all backend operations.
- Build a responsive Next.js UI with Tailwind CSS.
- Create an AI recommendation microservice using FastAPI and NumPy.
- Deploy the full stack on AWS with CI/CD.
- Achieve 80 % unit test coverage and basic performance benchmarks.

Functional Requirements

- User registration & login with JWT.
- Role based access control for admin, organizer, attendee.
- Organizers can create, edit, delete events.
- Attendees can browse events, book tickets, and download PDF tickets.
- Real time chat room per event using WebSocket.
- AI service returns personalized event suggestions.

Non Functional Requirements

- Passwords stored with bcrypt hashing.
- All APIs must validate input and return proper HTTP error codes.
- Page load time < 2 /seconds on standard broadband.
- Data must be backed up daily on AWS S3.
- Audit logs for admin actions.

Modules

Auth & User Management

Description: Handles registration, login, JWT issuance, and role checks.

Features

- Register
- Login
- Password reset
- Role verification

Event Management

Description: CRUD for events, image upload, and scheduling.

Features

- Create event
- Edit event
- Delete event
- List events

Ticketing

Description: Booking flow, seat allocation, PDF ticket generation.

Features

- Book ticket
- Cancel booking
- Generate PDF

Chat Service

Description: WebSocket based real time chat per event.

Features

- Join room
- Send message
- Receive messages

AI Recommendation

Description: FastAPI service that scores events using simple collaborative filtering.

Features

- Compute scores
- Return top 3 events

Admin Dashboard

Description: Analytics and audit log view for admins.

Features

- View metrics
- Search logs
- User management

Implementation Plan

Foundation

Objective: Set up repo, CI pipeline, and basic project scaffolding.

Milestone: Monorepo with separate backend and frontend directories, GitHub Actions CI.

Tasks

- Initialize Git repo with Lerna/Yarn workspaces
- Configure ESLint/Prettier for JS/TS
- Create CI workflow for lint and test
- Provision AWS EC2 for dev environment

Deliverables

- Repo structure
- CI config

- Dev server

Core Backend

Objective: Build authentication, user, and event APIs in Node/Express.

Milestone: All CRUD endpoints secured with JWT.

Tasks

- Design MongoDB schemas for users and events
- Implement JWT login & registration
- Create REST endpoints for event CRUD
- Add input validation middleware

Deliverables

- Auth API
- Event API
- Postman collection

Core Frontend

Objective: Develop UI for registration, login, and event browsing.

Milestone: Responsive Next.js pages with Tailwind styling.

Tasks

- Create pages: /login, /register, /events
- Integrate API calls with Axios & Promise.all
- Implement role based route guards
- Add PDF ticket download button

Deliverables

- Login page
- Event list UI
- PDF export

AI Recommendation Service

Objective: Add Python FastAPI microservice for event suggestions.

Milestone: Endpoint /recommend returns 3 event IDs.

Tasks

- Set up FastAPI project with virtualenv
- Implement simple collaborative filtering using NumPy
- Expose /recommend endpoint
- Dockerize service and push to AWS ECR

Deliverables

- AI service Docker image
- API docs

Testing & Security

Objective: Write unit, integration, and security tests; harden the system.

Milestone: 80 % coverage and OWASP based security checks.

Tasks

- Add Jest tests for backend routes
- Add React Testing Library tests for UI
- Perform JWT tampering tests
- Run npm audit and fix vulnerabilities

Deliverables

- Test reports
- Security audit log

Deployment & Optimization

Objective: Deploy full stack on AWS and tune performance.

Milestone: Live site with CI/CD, monitoring, and daily backups.

Tasks

- Create Docker Compose for all services
- Set up AWS Elastic Beanstalk for Node app
- Configure CloudWatch alarms
- Implement caching of event list with Redis (optional)
- Run load test with k6

Deliverables

- Production URL
- Monitoring dashboard
- Backup script

Evaluation Criteria

Correctness

Description: All core features work as specified

Weight: 30

Code Quality

Description: Readable, modular, follows lint rules

Weight: 15

Security

Description: Proper auth, input validation, password hashing

Weight: 15

Testing

Description: Coverage "e80 /% and meaningful test cases

Weight: 15

Documentation

Description: README, API docs, SRS

Weight: 10

Deployment

Description: CI/CD pipeline and live AWS deployment

Weight: 15

Deliverables

- GitHub repository with monorepo structure
- README with setup and deployment instructions
- Postman collection for backend APIs
- Swagger/OpenAPI spec for all services
- Dockerfiles and docker compose.yml
- Live URL and monitoring dashboard

Api Endpoints

POST

Path: /api/auth/register

Purpose: Create new user

POST

Path: /api/auth/login

Purpose: Issue JWT

GET

Path: /api/events

Purpose: List public events

POST

Path: /api/events

Purpose: Create event (organizer)

POST

Path: /api/tickets

Purpose: Book ticket

GET

Path: /api/recommend/:userId

Purpose: AI event suggestions

Database Design

Item 1

Entity: User

Fields

- _id
- name
- email
- passwordHash
- role
- createdAt

Item 2

Entity: Event

Fields

- _id
- title
- description
- date
- location
- organizerId
- capacity

Item 3

Entity: Ticket

Fields

- _id
- eventId
- userId
- purchaseDate
- status

Item 4

Entity: ChatMessage

Fields

- _id
- eventId
- userId
- message
- timestamp

Item 5

Entity: AuditLog

Fields

- _id
- adminId
- action
- targetId
- timestamp

Testing Plan

- Unit tests for auth middleware (Jest)
- Integration tests for event CRUD (Supertest)
- React component tests for forms (React Testing Library)
- API contract tests for AI service (pytest)
- Security test: JWT expiration and revocation
- Performance test: 100 concurrent event list requests (k6)

Student Fit: Rahul already knows JavaScript, TypeScript, React, Next.js, Node.js, Express, Tailwind, MongoDB, JWT and role based auth. This project lets him apply those skills while adding a Python FastAPI AI microservice, MySQL, AWS deployment, and automated testing – a realistic progression for a junior developer over a year.

Student Id: 6ab2576039d4e38828d96a4e

Status: active